

Correction — TP — Prédire les situations nécessitant une intervention

1 à 4. Charger et explorer les données

```
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree

df = pd.read_csv("interventions.csv")
df.head()
```

	temperature	humidity	wind_speed	fine_particles	visibility	traffic_level	intervention
0	17.4	90.5	10.4	32.4	6.5	faible	non
1	6.7	83.4	8.1	29.6	8.0	moyen	oui
2	21.0	59.2	10.9	20.1	11.6	moyen	non
3	22.5	47.5	16.6	38.7	9.4	fort	oui
4	-0.6	25.5	17.4	37.1	3.7	fort	oui

```
print("Nombre d'observations et de colonnes :", df.shape)
print("Colonnes :", df.columns.tolist())
df.info()
df.describe()
```

```

Nombre d'observations et de colonnes : (500, 7)
Colonnes : ['temperature', 'humidity', 'wind_speed', 'fine_particles', 'visibility', 'traffic_level', 'intervention']
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   temperature           500 non-null    float64
1   humidity              500 non-null    float64
2   wind_speed            500 non-null    float64
3   fine_particles        500 non-null    float64
4   visibility            500 non-null    float64
5   traffic_level         500 non-null    object  
6   intervention          500 non-null    object  
dtypes: float64(5), object(2)
memory usage: 27.5+ KB

```

	temperature	humidity	wind_speed	fine_particles	visibility
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	14.893000	69.261200	12.031400	30.573800	8.144800
std	7.678417	15.000961	7.402464	14.432922	2.554479
min	-5.500000	25.000000	0.300000	5.000000	1.000000
25%	9.675000	59.125000	6.700000	20.250000	6.400000
50%	15.050000	70.150000	10.700000	30.800000	8.000000
75%	19.700000	78.900000	16.200000	39.900000	9.925000
max	38.300000	100.000000	35.000000	72.900000	15.000000

Le jeu contient 500 observations et 7 colonnes : cinq variables numériques, une variable catégorielle (`traffic_level`) et la cible (`intervention`). Le dataset fourni contient 323 observations non et 177 observations oui. Les classes sont donc déséquilibrées, sans être extrêmement déséquilibrées.

```

df["intervention"].value_counts()
df["intervention"].value_counts(normalize=True)

```

```

intervention
non      0.646
oui      0.354
Name: proportion, dtype: float64

```

La répartition des classes doit être vérifiée avant l'entraînement : une accuracy élevée peut être obtenue en prédisant trop souvent la classe majoritaire.

5 à 7. Préparer les variables

```
X = df.drop(columns="intervention")
y = df["intervention"]

print("Dimensions de X :", X.shape)
print("Dimensions de y :", y.shape)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

```
Dimensions de X : (500, 6)
Dimensions de y : (500,)
```

`stratify=y` conserve approximativement la même proportion de `oui` et de `non` dans les jeux d'entraînement et de test. Sans cette option, une petite base de test pourrait être peu représentative.

Les colonnes sont réparties ainsi :

```
numeric_features = [
    "temperature",
    "humidity",
    "wind_speed",
    "fine_particles",
    "visibility",
]

categorical_features = ["traffic_level"]
```

L'encodage one-hot transforme les modalités textuelles en colonnes numériques distinctes. Par exemple, `traffic_level` devient des indicatrices comme `traffic_level_faible`, `traffic_level_moyen` et `traffic_level_fort`. Cela évite d'introduire un ordre artificiel entre les catégories.

8 à 12. Construire et évaluer le premier arbre

```
preprocessor = ColumnTransformer(  
    transformers=[  
        (  
            "categorical",  
            OneHotEncoder(handle_unknown="ignore"),  
            categorical_features,  
        ),  
    ],  
    remainder="passthrough",  
)  
  
model = Pipeline(  
    steps=[  
        ("preprocessor", preprocessor),  
        ("classifier", DecisionTreeClassifier(random_state=42)),  
    ]  
)  
  
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)  
  
print("Prédictions :", y_pred[:20])  
print("Valeurs réelles :", y_test.iloc[:20].to_numpy())  
print(f"Accuracy : {accuracy_score(y_test, y_pred):.2%}")  
print(classification_report(y_test, y_pred, zero_division=0))
```

```
Prédictions : ['non' 'non' 'non' 'non' 'oui' 'non' 'oui' 'non' 'oui' 'non' 'oui' 'non'  
              'non' 'non' 'oui' 'oui' 'oui' 'non' 'oui' 'non']  
Valeurs réelles : ['non' 'non' 'non' 'non' 'oui' 'non' 'oui' 'non' 'non' 'non' 'oui' 'non'  
                  'non' 'non' 'non' 'oui' 'oui' 'oui' 'non' 'non']
```

Accuracy : 60.00%

	precision	recall	f1-score	support
non	0.68	0.74	0.71	65
oui	0.41	0.34	0.38	35
accuracy			0.60	100
macro avg	0.54	0.54	0.54	100
weighted avg	0.58	0.60	0.59	100

Pour compter les bonnes prédictions affichées :

```
comparison = pd.DataFrame({
    "reelle": y_test.iloc[:20].to_numpy(),
    "predite": y_pred[:20],
})
comparison["correcte"] = comparison["reelle"] == comparison["predite"]
comparison["correcte"].sum()
```

```
np.int64(16)
```

L'accuracy doit être interprétée avec le rapport de classification. La précision mesure la fiabilité des prédictions d'une classe, tandis que le rappel mesure la proportion de cette classe correctement retrouvée. Pour une intervention de terrain, le rappel de la classe oui peut être particulièrement important.

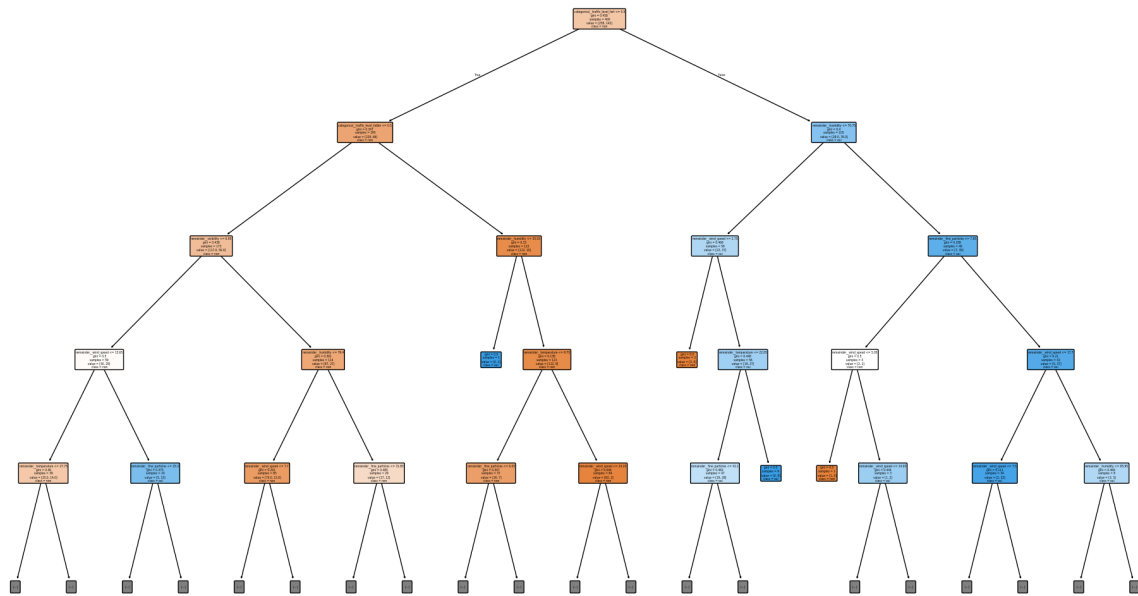
13 à 18. Visualiser et comprendre l'arbre

```
trained_tree = model.named_steps["classifier"]
feature_names = model.named_steps["preprocessor"].get_feature_names_out()

print("Profondeur :", trained_tree.get_depth())
print("Nombre de feuilles :", trained_tree.get_n_leaves())

plt.figure(figsize=(24, 14))
plot_tree(
    trained_tree,
    feature_names=feature_names,
    class_names=trained_tree.classes_,
    filled=True,
    rounded=True,
    max_depth=4,
)
plt.show()
```

```
Profondeur : 13
Nombre de feuilles : 83
```



`max_depth=4` limite uniquement l'affichage. L'arbre utilisé pour les prédictions reste l'arbre complet.

Pour examiner le chemin suivi par une observation de test :

```
observation = X_test.iloc[[0]]
observation_transformed = model.named_steps["preprocessor"].transform(observation)
node_indicator = trained_tree.decision_path(observation_transformed)
leaf_id = trained_tree.apply(observation_transformed)[0]

nodes = node_indicator.indices[node_indicator.indptr[0]:node_indicator.indptr[1]]
print("Observation :")
print(observation)
print("Nœuds traversés :", nodes)
print("Feuille atteinte :", leaf_id)
print("Classe prédite :", model.predict(observation)[0])
```

```
Observation :
   temperature  humidity  wind_speed  fine_particles  visibility \
197         11.8      55.5         9.5          49.4          7.8

   traffic_level
197         faible
```

Nœuds traversés : [0 1 89 91 107 108 112]
Feuille atteinte : 112
Classe prédite : non

L'arbre ne comprend pas la pollution au sens causal. Il recherche des seuils qui séparent efficacement les classes dans les données disponibles. Une règle comme `fine_particles <= 32.5` est utile pour prédire sur ce dataset, mais ne prouve pas que cette variable est la cause directe de l'intervention.

Critère de Gini

Pour deux classes :

$$Gini = 1 - p_{oui}^2 - p_{non}^2$$

```
situations = {
    "A": (20, 0),
    "B": (15, 5),
    "C": (10, 10),
}

for name, (yes_count, no_count) in situations.items():
    total = yes_count + no_count
    p_yes = yes_count / total
    p_no = no_count / total
    gini = 1 - p_yes**2 - p_no**2
    print(f"Situation {name} : {gini:.2f}")
```

Situation A : 0.00
Situation B : 0.38
Situation C : 0.50

Résultats :

Situation	Gini
A : 20 oui, 0 non	0,00
B : 15 oui, 5 non	0,38
C : 10 oui, 10 non	0,50

La situation A est la plus pure. La situation C est la plus mélangée. L'arbre recherche des séparations qui diminuent l'impureté des nœuds enfants.

19 à 23. Surapprentissage et profondeur

```
train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)

print(f"Accuracy entraînement : {accuracy_score(y_train, train_predictions):.2%}")
print(f"Accuracy test : {accuracy_score(y_test, test_predictions):.2%}")
```

Accuracy entraînement : 100.00%

Accuracy test : 60.00%

Une accuracy presque parfaite sur l'entraînement, nettement supérieure à celle du test, indique un risque de surapprentissage. L'arbre mémorise alors des détails du jeu d'entraînement qui se généralisent mal.

```
def build_tree_model(max_depth=None, criterion="gini"):
    return Pipeline(
        steps=[
            ("preprocessor", preprocessor),
            (
                "classifier",
                DecisionTreeClassifier(
                    max_depth=max_depth,
                    criterion=criterion,
                    random_state=42,
                ),
            ),
        ],
    )

model_depth_3 = build_tree_model(max_depth=3)
model_depth_3.fit(X_train, y_train)

print(
    "Train :",
    f"{accuracy_score(y_train, model_depth_3.predict(X_train)):.2%}",
)
print(
    "Test :",
    f"{accuracy_score(y_test, model_depth_3.predict(X_test)):.2%}",
)
```

Train : 77.25%
Test : 74.00%

Un arbre de profondeur 3 est plus simple à visualiser et à expliquer. Il peut toutefois être trop limité si la relation entre les variables et la cible est complexe.

```
depths = [1, 2, 3, 4, 5, 7, 10, None]
results = []

for depth in depths:
    current_model = build_tree_model(max_depth=depth)
    current_model.fit(X_train, y_train)
    train_pred = current_model.predict(X_train)
    test_pred = current_model.predict(X_test)

    results.append({
        "max_depth": depth,
        "train_accuracy": accuracy_score(y_train, train_pred),
        "test_accuracy": accuracy_score(y_test, test_pred),
    })

results_df = pd.DataFrame(results)
results_df
```

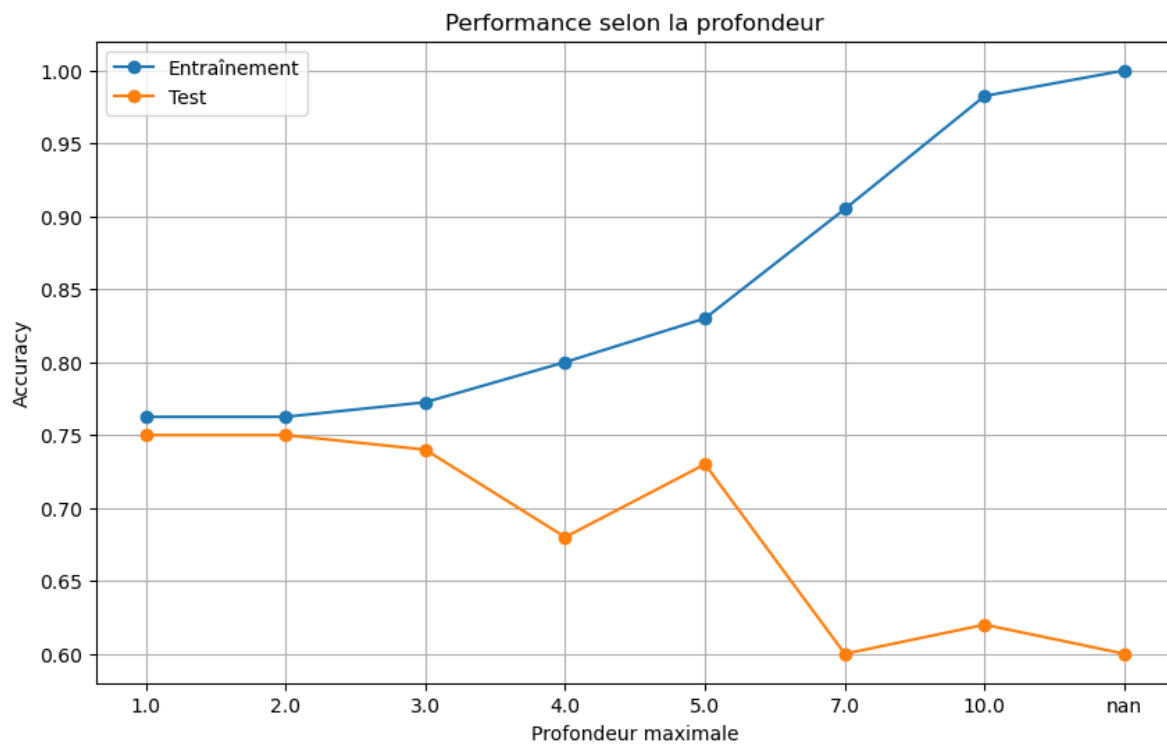
	max_depth	train_accuracy	test_accuracy
0	1.0	0.7625	0.75
1	2.0	0.7625	0.75
2	3.0	0.7725	0.74
3	4.0	0.8000	0.68
4	5.0	0.8300	0.73
5	7.0	0.9050	0.60
6	10.0	0.9825	0.62
7	NaN	1.0000	0.60

```
plt.figure(figsize=(10, 6))
plt.plot(
    results_df["max_depth"].astype(str),
    results_df["train_accuracy"],
    marker="o",
    label="Entraînement",
```

```

)
plt.plot(
    results_df["max_depth"].astype(str),
    results_df["test_accuracy"],
    marker="o",
    label="Test",
)
plt.xlabel("Profondeur maximale")
plt.ylabel("Accuracy")
plt.title("Performance selon la profondeur")
plt.legend()
plt.grid()
plt.show()

```



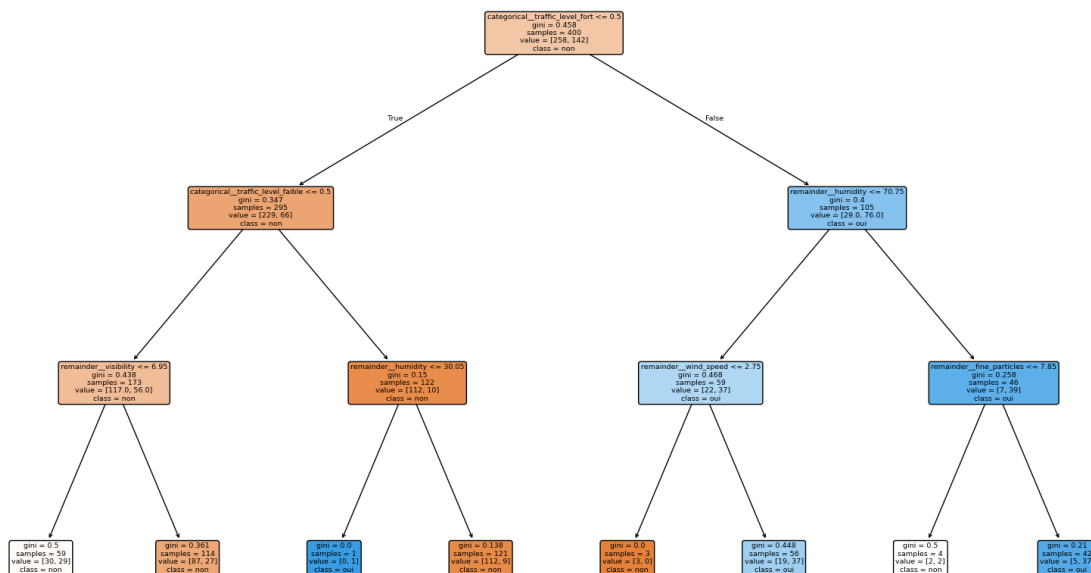
Lorsque la profondeur augmente, l'accuracy d'entraînement tend à augmenter, tandis que l'accuracy de test peut plafonner puis diminuer. On choisit la profondeur au voisinage du meilleur compromis, en privilégiant un modèle simple et stable plutôt que la meilleure accuracy ponctuelle du jeu de test.

```

trained_tree = model_depth_3.named_steps["classifier"]
feature_names = model_depth_3.named_steps["preprocessor"].get_feature_names_out()

plt.figure(figsize=(20, 12))
plot_tree(
    trained_tree,
    feature_names=feature_names,
    class_names=trained_tree.classes_,
    filled=True,
    rounded=True,
)
plt.show()

```



24. Réflexion

Une erreur dans les données d'entraînement peut créer une séparation artificielle. Un arbre très profond peut lui accorder trop d'importance et créer une branche spécifique pour cette observation. Un arbre limité en profondeur est généralement moins sensible à ce type de bruit.

L'arbre est intéressant pour un métier car ses décisions peuvent être présentées sous forme de règles : seuil sur une variable, branche suivie, puis classe de la feuille. Cette interprétabilité ne garantit toutefois ni une causalité ni une absence de biais.

25. Comparer gini et entropy

```
criteria_results = []

for criterion in ["gini", "entropy"]:
    current_model = build_tree_model(max_depth=3, criterion=criterion)
    current_model.fit(X_train, y_train)

    criteria_results.append({
        "criterion": criterion,
        "train_accuracy": accuracy_score(
            y_train,
            current_model.predict(X_train),
        ),
        "test_accuracy": accuracy_score(
            y_test,
            current_model.predict(X_test),
        ),
    })

criteria_df = pd.DataFrame(criteria_results)
criteria_df
```

	criterion	train_accuracy	test_accuracy
0	gini	0.7725	0.74
1	entropy	0.7625	0.77

Les deux critères mesurent l'impureté avec des formules différentes. Ils peuvent produire des arbres légèrement différents, mais aucun n'est systématiquement meilleur. Le choix doit être fait sur une validation fiable et en tenant compte de la simplicité du modèle.

26. Choisir une profondeur

```

depths = range(1, 16)
results = []

for depth in depths:
    current_model = build_tree_model(max_depth=depth)
    current_model.fit(X_train, y_train)
    train_pred = current_model.predict(X_train)
    test_pred = current_model.predict(X_test)

    results.append({
        "max_depth": depth,
        "train_accuracy": accuracy_score(y_train, train_pred),
        "test_accuracy": accuracy_score(y_test, test_pred),
    })

results_df = pd.DataFrame(results)
results_df

```

	max_depth	train_accuracy	test_accuracy
0	1	0.7625	0.75
1	2	0.7625	0.75
2	3	0.7725	0.74
3	4	0.8000	0.68
4	5	0.8300	0.73
5	6	0.8700	0.61
6	7	0.9050	0.60
7	8	0.9400	0.61
8	9	0.9650	0.62
9	10	0.9825	0.62
10	11	0.9925	0.60
11	12	0.9975	0.62
12	13	1.0000	0.60
13	14	1.0000	0.60
14	15	1.0000	0.60

Pour une sélection rigoureuse, on utiliserait une validation croisée plutôt que de choisir directement la profondeur qui maximise un seul jeu de test. Une profondeur légèrement moins performante mais plus simple et plus stable peut être préférable.

27. Challenge métier

- Le cas A est un **faux négatif** : une intervention nécessaire n'est pas déclenchée. Il peut entraîner un risque opérationnel ou environnemental important.
- Le cas B est un **faux positif** : une intervention est déclenchée inutilement. Il peut entraîner un coût, une mobilisation inutile des agents ou une perte de confiance.

Ces erreurs n'ont pas nécessairement le même coût. L'accuracy les compte de la même manière et ne suffit donc pas. Il faut examiner la matrice de confusion, le rappel de la classe oui, la précision de cette classe et, si nécessaire, utiliser des poids de classe ou ajuster le seuil de décision.

28. Importance des variables

```
trained_tree = model_depth_3.named_steps["classifier"]
feature_names = model_depth_3.named_steps["preprocessor"].get_feature_names_out()

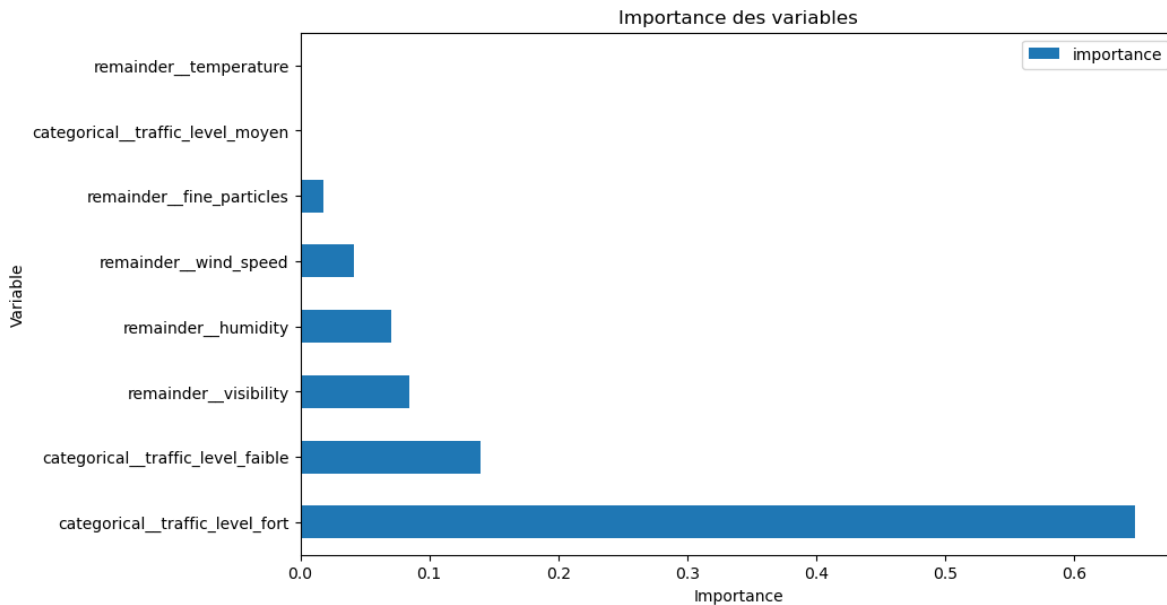
feature_importance_df = pd.DataFrame({
    "feature": feature_names,
    "importance": trained_tree.feature_importances_,
}).sort_values("importance", ascending=False)

feature_importance_df
```

	feature	importance
1	categorical__traffic_level_fort	0.646597
0	categorical__traffic_level_faible	0.139590
7	remainder__visibility	0.084200
4	remainder__humidity	0.070412
5	remainder__wind_speed	0.041504
6	remainder__fine_particles	0.017697
2	categorical__traffic_level_moyen	0.000000
3	remainder__temperature	0.000000

```
feature_importance_df.head(10).plot(
    x="feature",
    y="importance",
    kind="barh",
    figsize=(10, 6),
```

```
)
plt.xlabel("Importance")
plt.ylabel("Variable")
plt.title("Importance des variables")
plt.show()
```



Les variables les plus importantes sont celles utilisées pour réaliser les séparations les plus utiles dans cet arbre. Le classement exact dépend de la profondeur et de la séparation train/test. Une importance élevée indique une association utile pour les prédictions, pas une relation causale : elle peut refléter une variable indirecte, une corrélation ou un biais du jeu de données.

29. Synthèse

- Un arbre de décision fonctionne en appliquant successivement des règles de séparation.
- Le critère de Gini mesure le mélange des classes dans un nœud.
- Un arbre trop profond risque de surapprendre les données d'entraînement.
- `max_depth` limite la profondeur et contrôle la complexité du modèle.
- Le principal avantage d'un arbre est son interprétabilité.
- Ses principales limites sont la sensibilité au bruit, le surapprentissage et l'instabilité face à de petites variations des données.

Bilan

L'accuracy ne doit pas être utilisée seule. Une évaluation adaptée doit tenir compte de la répartition des classes et du coût métier des faux positifs et des faux négatifs. Pour ce problème, un modèle compréhensible et correctement évalué peut aider à prioriser les interventions, mais il ne remplace pas l'expertise des services opérationnels.